

INFORME FINAL DESAFIO I

SWEET CRUSH

Análisis, algoritmos, problemas de desarrollo y evolución de la solución

Jose Gabriel Giraldo Florez

Informática II Semestre 2026-1

Septiembre 2026

Índice

1. Análisis del problema	2
2. Esquema de tareas y arquitectura de módulos	2
3. Algoritmos implementados	3
3.1. Ventana de 16bits (extraerBits3/insertarBits3)	3
3.2. Detección de corridas (detectarHorizontales/detectarVerticales)	3
3.3. Gravedad (aplicarGravedad)	4
3.4. Bucle de cascada (procesarCascadas)	4
3.5. Re-socialización de estructura (agregarFila/eliminarFila/agregarColumna/eliminarColumna)	4
3.6. Regla del 65 % (decidirRedimensionar)	4
3.7. Validación de entrada (main.cpp)	4
4. Problemas de desarrollo enfrentados	4
4.1. Riesgo de doble conteo en fichas de cruce horizontal-vertical	4
4.2. Pérdida de un operador al refactorizar	5
4.3. Ambigüedad en el registro de cascadas	5
4.4. Desalineación visual en tableros de gran magnitud	5
4.5. Entrada no numérica sin validar	5
5. Evolución de la solución	5

1. Análisis del problema

El reto central del Desafío I se basa en la forma de representar el juego Sweet crush, donde cada ficha del tablero debe almacenarse exactamente en 3 bits, empaquetados de forma continua en bloque de memoria dinámica, sin relleno para alinear cada ficha a un byte. Esto implica ciertas consecuencias directas sobre el diseño.

- Una ficha puede quedar completamente contenida dentro de un byte, o repartida entre 2 bytes consecutivos, teniendo que resolver su acceso para ambos casos de forma unificada.
- El tablero no se puede desempaquetar nunca en una estructura auxiliar de un byte o un entero por posición; cada lectura o escritura debe pasar por operadores de bits en el momento que sea necesario.

Para la solución implementada se calcula para una ficha en (fila, columna), su posición lógica $pos = fila * columnas + columna$ y su bit inicial $pos * 3$, a partir de esto, $byte = bitInicial / 3$ y el desfase $= bitInicial \% 7$ (usando así corrimiento y mascara en lugar de $/$ y $\%$, ya que 8 es potencia de 2).

Para resolver el acceso sin tener que ramificar en los 3 escenarios posibles como lo son: (ficha en un byte, o repartida 2+1 / 1+2), se hace una combinación del byte actual y el siguiente para una ventana de 16 bits, extrayendo o insertando los 3 bits con corrimiento y mascara sobre la respectiva ventana.

Para la gestión de memoria se tuvo en cuenta bajo la restricción explicita del enunciado: la memoria reservada corresponde siempre a las dimensiones actuales, salvo al momento de eliminar filas o columnas, donde la reasignación física solo ocurre si el uso cae por debajo del 65 % de lo reservado. Por ende, para modificar la estructura (agregar/eliminar fila o columna en cualquier posición, no solo en los extremos), se decidió por re serializar el tablero completo en un nuevo buffer en vez de desplazar bits a mano caso por caso, ya que el empaquetado sin alineación por fila hace que una inserción o eliminación intermedia mueva la ubicación física de casi todas las fichas posteriores.

2. Esquema de tareas y arquitectura de módulos

Para el proyecto se separó el programa en 8 módulos independientes (.h/.cpp) más main.cpp, cada uno correspondiente a una responsabilidad explicita del enunciado para el cumplimiento del programa en su totalidad.

Módulo	Tarea
bits	Extracción/inserción de una ficha de 3 bits (base de todo el proyecto).
tablero	Crear/liberar el tablero (uso de memoria dinámica), obtener/establecer fichas.
fichas	Código de fichas, generación aleatoria, símbolo visual.
combinaciones	Detección horizontal/vertical, marcado, eliminación.
reorganización	Gravedad, relleno, bucle de cascadas y puntuación.
estructura	Agregar/eliminar fila o columna, regla del 65 %.
estado	Contadores de partida y puntuación.
visualización	Mostrar tablero (fichas/binario) y estadísticas.

El orden de implementación siguió las dependencias entre los módulos: primero se realizó bits debido a que es la base de la cual dependen todos los demás, luego se prosigue con tablero/fichas, visualización para tener el tablero visible desde temprano, se pasa a combinaciones, reorganización, estructura, estado y por último el programa principal main.cpp para terminar integrando en este todo en un menú de consola.

3. Algoritmos implementados

3.1. Ventana de 16bits (extraerBits3/insertarBits3)

Combina `tablero[byte]` y `tablero [byte + 1]` en una variable de 16 bits para leer o escribir los 3 bits de una ficha sin distinguirlo por su código si cruza la frontera entre byte. La escritura limpia la posición con `& ~mascara` y aplica el valor nuevo con `^`, usando de esta manera los 6 operadores de bits del enunciado (`&`, `|`, `^`, `~`, `«`, `»`) con un propósito real dentro del desarrollo.

3.2. Detección de corridas (detectarHorizontales/detectarVerticales)

Se basa en el recorrido de 2 punteros (inicio, c/f) que cuenta fichas iguales consecutivas; cada corrida ≥ 3 se marca en un arreglo auxiliar. Ambas direcciones escriben sobre el mismo marcador sin reiniciarlo, de modo que una ficha en cruce horizontal-vertical quede marcada una única vez.

3.3. Gravedad (aplicarGravedad)

Se usa una técnica de 2 punteros haciendo una lectura desde el fondo hacia arriba, escritura que solo avanza al encontrar una ficha real, que compacta cada columna hacia abajo sin necesidad de un arreglo auxiliar.

3.4. Bucle de cascada (procesarCascadas)

Repite detectar/eliminar/gravedad/relleno hasta que una vuelta no encuentre combinaciones nuevas, acumulando así fichas eliminadas, combinaciones y puntuación $(10 * \text{fichas} * \text{nivel}_{\text{cascada}}) \text{ en cada una de las vueltas.}$

3.5. Re-socialización de estructura (agregarFila/eliminarFila/agregarColumna)

Para estas funcionalidades se reserva un buffer del tamaño correcto y se recorre cada posición del tablero nuevo, mapeando de esta forma al origen en el tablero con ayuda de un operador ternario ($f < \text{posición} ? f: f-1, \text{ o } f+1 \text{ al eliminar}$).

3.6. Regla del 65 % (decidirRedimensionar)

Compara $\text{bits}_{\text{nuevos}} / (\text{bytes}_{\text{actuales}} * 8)$ contra 0,65 para decidir si vale la pena reasignar memoria a eliminar.

3.7. Validación de entrada (main.cpp)

Para las validaciones cada lectura de teclado usa bucles que revisa `cin.fail()` para tipo y rango antes de aceptar el valor, evitando bucles infinitos frente a entradas no numéricas.

4. Problemas de desarrollo enfrentados

4.1. Riesgo de doble conteo en fichas de cruce horizontal-vertical

Para esto eliminar directamente cada combinación a medida que se detecta se corría el riesgo de procesar dos veces una ficha que forma parte simultáneamente de una corrida horizontal y una vertical. Esto se le dio solución separando el proceso en 2 pasos importantes: primero marcar todas las posiciones de ambas direcciones sobre

el mismo arreglo auxiliar (sin reiniciar entre una detección y otra), y solo después recorrer el tablero eliminando lo marcado, garantizando de este modo que cada posición se procesa una única vez sin importar en cuantas combinaciones participe.

4.2. Pérdida de un operador al refactorizar

Al cambiar la función de insertarBits3 para incorporar $\wedge$ de forma genuina, la primera versión elimino accidentalmente el uso de $\sim$ que si tenía la implementación original. Resolviéndose con la combinación de ambos operadores en la misma línea ((ventana & $\sim$ mascara) $\wedge$ valorCorrido), en vez de sustituir uno por otro.

4.3. Ambigüedad en el registro de cascadas

Para el problema se pide, en secciones distintas, tanto el total de cascadas “durante la partida” como las cascadas “de la eliminación o jugada actual”. La primera versión realizada solo llevaba el conteo acumulado; pasando a agregar cascadasUltimaJugada, la cual se sobre escribe (no se acumula) en cada llamada a procesarCascadas.

4.4. Desalineación visual en tableros de gran magnitud

Al hacerse pruebas con 50 columnas, los encabezados numéricos de 2 dígitos desalineaban la cuadrícula frente a los símbolos de 1 sola posición. Lo cual se corrigió con ayuda de setw() (para permitir determinar el ancho que se ocuparía) de <iomanip> para forzar ancho fijo.

4.5. Entrada no numérica sin validar

Inicialmente el menú no contaba con comprobaciones para el estado cin tras cada lectura; una entrada de texto donde se esperaba un numero dejaba el flujo de entrada en estado de error permanente. Se resolvió revisando cin.fail() y limpiando el flujo (cin.clear() + cin.ignore()) antes de reintentar.

5. Evolución de la solución

El desarrollo avanza modulo por modulo, con commits independientes por cada uno para evidenciar el progreso de implementación:

1. Bits.h/bits.cpp, por ser la base de la cual depende directa o indirectamente todo lo demás, realizando inicialmente su implementación y de forma separada antes de la construcción.
2. Tablero y fichas, con una verificación visual temprana mediante la ayuda del módulo de visualización.
3. Combinaciones, cuya lógica se diseñó principalmente separando la etapa de detección y marcado de la etapa de eliminación, para la resolución del problema establecido de fichas en cruces horizontales-verticales.
4. Reorganización y estructura, dejando de esta forma para el final los componentes de mayor riesgo (regla del 65 %, inserciones intermedias) una vez el resto fuera validado.
5. Estado y main.cpp, integrando el menú completo.

Sobre esa base funcional se aplicaron refinamientos posteriores: se evaluó empaquetar en bits el arreglo auxiliar de combinaciones, y finalmente se revirtió esa decisión por preferir mantener esa parte de la lógica más simple; se reemplazaron las divisiones/módulos entre 8 por corrimientos y máscaras explícitos; se reforzó el uso genuino de los 6 operadores de bits en insertarBits3; y se agregaron las validaciones de tipo y rango descritas en la sección anterior. Cada uno de estos cambios se contrastó cuidadosamente contra el comportamiento esperado del resto del programa antes de aceptarlo, para asegurar que ninguna corrección introdujera una regresión.